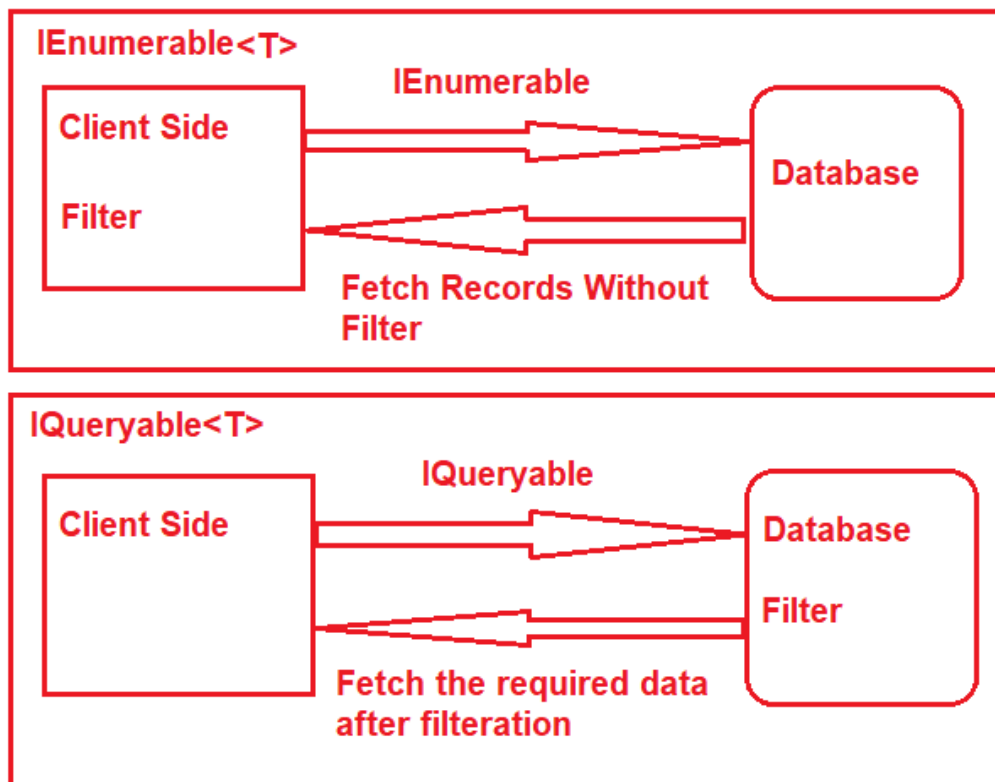


Differences Between IEnumerable and IQueryable in C#

In this article, I will discuss the **Differences Between IEnumerable and IQueryable in C#** with Examples. Please read our previous article before proceeding to this article, where we discussed the basics of **IEnumerable and IQueryable in C#** with Examples.

Differences Between IEnumerable and IQueryable in C#

The IEnumerable and IQueryable in C# are used to hold a collection of data and also to perform data manipulation operations such as filtering, ordering, grouping, etc., based on the business requirements. This article will show you the difference between IEnumerable and IQueryable in C# with Examples. For a better understanding, please have a look at the following image. As you can see, IEnumerable<T> fetches the record from the database without applying the filter. But IQueryable<T> fetches the record from the database by applying the filter.



Example to Understand the Differences Between IEnumerable and IQueryable in C#

In this demo, we will create a Console Application to retrieve the data from the SQL Server database using the Entity Framework Database First approach. We are going to fetch the following Student information from the Student table.

ID	FirstName	LastName	Gender
101	Steve	Smith	Male
102	Sara	Pound	Female
103	Ben	Stokes	Male
104	Jos	Butler	Male
105	Pam	Semi	Female

Please use the SQL script below to create and populate the Student table with the required test data.

```
-- Create the required Student table
CREATE TABLE Student
(
    ID INT PRIMARY KEY,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    Gender VARCHAR(50)
)
GO
-- Insert the required test data
INSERT INTO Student VALUES (101, 'Steve', 'Smith', 'Male')
INSERT INTO Student VALUES (102, 'Sara', 'Pound', 'Female')
INSERT INTO Student VALUES (103, 'Ben', 'Stokes', 'Male')
INSERT INTO Student VALUES (104, 'Jos', 'Butler', 'Male')
INSERT INTO Student VALUES (105, 'Pam', 'Semi', 'Female')
GO
```

Create a new Console application. Once you create the Console Application, add the ADO.NET Entity Data Model using the Database First Approach pointing to the above database.

Example to Understand the Use of IEnumerable in C#

Let us modify the Main method of the Program class as shown below. In the example below, we are fetching the top 2 students from the Students table where the gender is male. But we have split the LINQ query into two statements. The first statement contains the where method, and the second statement contains the Take method. Then, using a for each loop, we display the top 2 student information. Further, to see what SQL Statement was generated and executed on the database by Entity Framework, we are using **DbContext.Database.Log = Console.Write** statement that will log the SQL Script on the Console window.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
```

```

{
    using (StudentDBContext DBContext = new StudentDBContext())
    {
        //To See What SQL Generated By Entity Framework
        DBContext.Database.Log = Console.Write;
        //Fetch the Top 2 Records from the Students Database table where
        Male
        IEnumerable<Student> listStudents = DBContext.Students.Where(s
x.Gender == "Male");
        listStudents = listStudents.Take(2);
        Console.WriteLine("Top 2 Student Where Gender = Male");
        foreach (var std in listStudents)
        {
            Console.WriteLine(std.FirstName + " " + std.LastName);
        }
    }

    Console.ReadKey();
}
}
}

```

Here, we create the LINQ Query using IEnumerable. With the above code in place, run the application and see the output. You should get the following output.

```

Top 2 Student Where Gender = Male
Opened connection at 06-02-2023 21:02:07 +05:30
SELECT
    [Extent1].[ID] AS [ID],
    [Extent1].[FirstName] AS [FirstName],
    [Extent1].[LastName] AS [LastName],
    [Extent1].[Gender] AS [Gender]
FROM [dbo].[Student] AS [Extent1]
WHERE 'Male' = [Extent1].[Gender]
-- Executing at 06-02-2023 21:02:07 +05:30
-- Completed in 9 ms with result: SqlDataReader

Steve Smith
Ben Stokes
Closed connection at 06-02-2023 21:02:07 +05:30

```

SQL Query not contains the Top 2 Clause
The Query will Retrieve all the Male
Students from the database and then in-
memory it will take the top 2 students

As you can see in the above SQL Script, it will not use the TOP clause. So, here, it will fetch all the Male Students from SQL Server to in-memory, and then it will filter the data in memory.

Example to Understand the Use of IQueryable in C#

Let us modify the Main method of the Program class as shown below to use IQueryable. The following example does the same thing as the previous one, but here, we store the query in a variable of IQueryable<Student> type. For this, we are using the AsQueryable() method. We are also logging the generated SQL Statement on the Console window.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            using (StudentDBContext DBContext = new StudentDBContext())
            {
                //To See What SQL Generated By Entity Framework
                DBContext.Database.Log = Console.Write;
                //Fetch the Top 2 Records from the Students Database table where
                Male
                IQueryable<Student> listStudents = DBContext.Students
                    .AsQueryable()
                    .Where(x => x.Gender == "Male");
                listStudents = listStudents.Take(2);
                Console.WriteLine("Top 2 Student Where Gender = Male");
                foreach (var std in listStudents)
                {
                    Console.WriteLine(std.FirstName + " " + std.LastName);
                }

                Console.ReadKey();
            }
        }
    }
}

```

Here, we create the LINQ Query using IQueryable. With the above changes in place, now run the application and see the output. You should get the following output.

```

Top 2 Student Where Gender = Male
Opened connection at 06-02-2023 21:16:53 +05:30
SELECT TOP (2)
    [Extent1].[ID] AS [ID],
    [Extent1].[FirstName] AS [FirstName],
    [Extent1].[LastName] AS [LastName],
    [Extent1].[Gender] AS [Gender]
FROM [dbo].[Student] AS [Extent1]
WHERE 'Male' = [Extent1].[Gender]
-- Executing at 06-02-2023 21:16:53 +05:30
-- Completed in 11 ms with result: SqlDataReader

Steve Smith
Ben Stokes
Closed connection at 06-02-2023 21:16:53 +05:30

```

SQL Query Contains the Top N Clause which means now the data is filtered in the database and only the top 2 records will be returned.

As you can see in the above image, it includes the TOP (2) clause in the SQL Script and then fetches the data from the database. That means the filtering is now happening on the database side. With this in mind, let us discuss the differences between IEnumerable and IQueryable in C#.

IEnumerable vs. IQueryable in C#

IEnumerable and IQueryable are both interfaces in C# that are used for data manipulation and query operations, but they serve different purposes and are used in different contexts. Understanding their differences is important for efficient and effective data operations in C#. Here's a comparison:

Namespace:

- **IEnumerable:** It is defined in the System.Collections namespace.
- **IQueryable:** It is defined in the System.Linq namespace.

Purpose and Data Sources:

- **IEnumerable:** Primarily used for querying and manipulating in-memory collections, such as arrays, lists, and IEnumerable-compatible collections. Designed for querying data that is already in memory.
- **IQueryable:** Used for querying data from external data sources that may not be in memory, such as databases, web services, or remote data stores. Designed for querying data that resides outside of the application's memory.

Data Source:

- **IEnumerable:** It can be used with any in-memory data collection, like arrays, lists, etc.
- **IQueryable:** It is typically used for remote data sources like databases, especially with ORM frameworks like Entity Framework.

Querying Capability:

- **IEnumerable:** It supports LINQ-to-Objects, meaning it can execute LINQ queries on in-memory collections.
- **IQueryable:** It supports LINQ-to-Entities, meaning it can translate LINQ queries into database-specific query languages (like SQL for relational databases).

Execution Location:

- **IEnumerable:** Operations are executed in memory. All data is retrieved from the collection, and subsequent operations are performed in memory.
- **IQueryable:** Operations are typically translated into a query language (e.g., SQL for databases) and executed on the data source. This allows for server-side processing and optimization.

Lazy Evaluation:

- **IEnumerable:** Supports lazy evaluation. Operations are only executed when the collection is enumerated (e.g., in a for each loop).

- **IQueryable**: Also supports lazy evaluation. Queries are not executed until you enumerate the results, allowing for efficient resource use.

Query Translation:

- **IEnumerable**: LINQ methods are executed in memory on the entire data set. Filtering, sorting, and other operations are performed locally.
- **IQueryable**: LINQ methods are translated into a query language specific to the data source (e.g., SQL) and executed on the data source. This allows for efficient server-side operations.

Suitable Use Cases:

- **IEnumerable**: Suitable for working with small to moderate-sized in-memory collections where the entire data set can fit in memory. Typically used for in-memory data manipulation and querying.
- **IQueryable**: Suitable for working with large data sets or data sources external to the application, such as databases. It is ideal for offloading query execution to the data source, enabling efficient database querying.

Common Use Cases:

- **IEnumerable**: Used for querying and manipulating in-memory collections, filtering data, and applying transformations.
- **IQueryable**: Used with Entity Framework for querying databases using LINQ, where queries are translated into SQL statements.

Examples:

- **IEnumerable**: Querying and filtering a list of objects in memory. Working with arrays or lists.
- **IQueryable**: Querying a database using Entity Framework. Querying data from a web service or remote data source.

Performance:

- **IEnumerable**: When querying data sources like databases, IEnumerable can be less efficient because it retrieves all the data from the database and then filters records in the client's memory.
- **IQueryable**: IQueryable can be more efficient for large datasets and database operations since it sends query expressions to the database, which then returns the filtered result.

Choosing Between Them

1. Use IEnumerable for in-memory data collections or when dealing with small data sets.
2. Use IQueryable when querying large data sets or remote data sources like databases, especially when you need efficient querying and data retrieval.

In the next article, I will discuss the [LINQ Extension Methods in C#](#) with Examples. In this article, I explain the **Differences Between IEnumerable and IQueryable in C#**. I hope this article gives you a very good understanding of the Differences Between IEnumerable and IQueryable in C# with Examples.